DOCKSMITH

An In-Depth Analysis of a Custom Container Engine Implementation

Surabhi M Smrithi A S Gahnavi B Tanuu Shree M

PES1UG23AM325 PES1UG23AM306 PES1UG23AM900 PES1UG23AM336

PESU CSE(AI&ML) JAN – MAY 2026

Abstract: *This document presents the comprehensive technical documentation for Docksmith, a lightweight container runtime and build system developed in Go. The project aims to deconstruct the internal mechanics of containerization by implementing immutable layering, deterministic caching, and kernel-level process isolation from first principles. By utilizing Linux system calls such as chroot and clone, Docksmith provides a secure execution environment that replicates the core functionality of industry-standard runtimes while maintaining a minimal architectural footprint.*

1. INTRODUCTION

Modern cloud computing relies heavily on the abstraction of physical hardware through containerization. While tools like Docker and Podman are widely adopted, their internal complexities often obscure the fundamental system-level interactions that enable process isolation. Docksmith was developed to bridge this knowledge gap. The project provides a custom implementation of an image build pipeline and an isolated execution environment, ensuring that each phase of the container lifecycle—from the parsing of a Docksmithfile to the cleanup of image layers—is handled via direct interaction with the Linux kernel.

The primary objectives of this project are twofold: first, to implement a content-addressable storage system that handles immutable filesystem layers; and second, to demonstrate the application of Linux namespaces and root directory pivoting to achieve secure process sandboxing.

2. SYSTEM OVERVIEW AND FUNCTIONAL SCOPE

Docksmith is a CLI-driven tool that manages the entire lifecycle of a container image. It is designed to be a "thick-client" runtime, where the binary itself contains the logic for parsing, building, and executing containers without requiring a background daemon process.

2.1 The Docksmithfile Specification

The system interprets a domain-specific language (DSL) called the Docksmithfile. This file contains a series of instructions that the build engine processes sequentially. The supported instructions include:

- **FROM:** Defines the base image, which serves as the foundation for the filesystem stack.
- **WORKDIR:** Sets the execution context for all subsequent build steps and the final runtime environment.
- **COPY:** Ingests files from the host machine into the image filesystem, maintaining original permissions and directory structures.
- **ENV:** Injects key-value pairs into the process environment variables.
- **RUN:** Executes a command within a temporary isolated environment to capture filesystem modifications.
- **CMD:** Defines the default entrypoint and arguments for the container execution phase.

2.2 Directory Structure and State Management

All persistent data is stored in a centralized directory, typically located at `/root/.docksmith/`. This directory is partitioned into four primary zones:

Zone	Description	Content Type
Images	JSON Manifests	Metadata regarding layer order and config.
Layers	Tar Archives	Content-addressed filesystem deltas.
Cache	Key-Value Index	Mappings between instruction hashes and layers.
Temp	Extracted RootFS	Ephemeral directories for container execution.

3. THE BUILD ENGINE: IMMUTABLE LAYERING ANDCACHING

At the heart of Docksmith is a sophisticated build engine that transforms high-level instructions into a stack of immutable filesystem layers. Each layer represents a "diff" or a delta from the previous state.

3.1 Content-Addressable Storage (CAS)

Docksmith employs a CAS model. When a layer is created, its contents are archived into a tarball. The SHA-256 hash of this tarball becomes its unique identifier. This approach provides two significant advantages: deduplication (if two images share the same layer, only one copy is stored) and integrity verification (any corruption in the layer is immediately detectable via hash mismatch).

3.2 Achieving Determinism

A persistent challenge in distributed build systems is non-determinism. To ensure that the same input always produces the same layer hash, Docksmith implements the following safeguards:

1. **Path Sorting:** Files are added to the tar archive in a strict lexicographical order.
2. **Epoch Normalization:** File modification and access times are reset to January 1, 1970 (Unix Epoch). This prevents timestamps from altering the hash of otherwise identical files.
3. **Ownership Masking:** User and Group IDs are normalized to ensure compatibility across different host machines.

3.3 The Caching Mechanism

The engine utilizes a cascading cache. Before executing any instruction, Docksmith generates a cache key based on the parent layer's digest combined with the current instruction's string and arguments. If the engine finds a match in the cache index, it avoids the expensive overhead of file copying or command execution, instantly referencing the pre-existing layer digest.

4. THE RUNTIME: PROCESS ISOLATION AND SANDBOXING

The runtime module is responsible for the secure execution of containerized processes. This is achieved by manipulating the execution context of the child process using native Linux kernel primitives.

4.1 Linux Namespaces

Docksmith utilizes the `unshare`-equivalent functionality via Go's `SysProcAttr` to isolate various system resources:

- **PID Namespace:** The container process becomes PID 1 within its own tree. It cannot see or kill processes running on the host or in other containers.
- **Mount Namespace:** This allows the runtime to change the root filesystem (chroot) and mount temporary filesystems (like proc) without affecting the host's mount table.
- **UTS Namespace:** Provides isolation for the system hostname and domain name.

4.2 Filesystem Assembly and Chroot

When a container is initiated, the runtime performs the following steps:

1. Creates a temporary directory to serve as the rootfs.
2. Iteratively extracts the stack of tar layers into this directory, starting from the base image and moving upwards.
3. Uses the `chroot` system call to set this temporary directory as the root for the new process.
4. Mounts the `/proc` filesystem inside the sandbox to enable standard process monitoring tools (like ps or top) to function correctly.

```
// Example of the low-level isolation configuration in Go cmd.SysProcAttr = &syscall.SysProcAttr{ Cloneflags: syscall.CLONE_NEWNS | syscall.CLONE_NEWPID | syscall.CLONE_NEWUTS, Chroot: rootfsPath, }
```

5. IMPLEMENTATION CHALLENGES AND SOLUTIONS

During the development of Docksmith, several low-level systems programming challenges were encountered and resolved.

5.1 The Symlink Problem

Initial versions of the engine failed to correctly handle symbolic links, particularly in the base Alpine image where common utilities (like /bin/ls) are symlinks to BusyBox. This resulted in "file not found" errors during execution. The solution involved extending the `util/tar.go` module to explicitly recognize `tar.TypeSymlink` and recreate the links using `os.Symlink` during extraction.

5.2 Layer Diffing Algorithm

Implementing the `RUN` instruction required a method to capture only the changes made by a command. A naive copy of the entire rootfs would be inefficient. Instead, Docksmith implements a "Snapshot-and-Diff" strategy. The engine records the state of the filesystem before the command runs and compares it to the state after completion. Only files that are new or have modified sizes/timestamps are captured into the new layer.

6. CONCLUSION

The Docksmith project successfully demonstrates that the complex abstractions of modern containerization can be decomposed into a manageable set of system-level operations. By prioritizing determinism in the build phase and strict isolation in the runtime phase, Docksmith provides a secure and efficient platform for running containerized workloads. The resulting system is a robust tool that provides full transparency into the process of image creation and process sandboxing.

Final Assessment: The system satisfies all project requirements, including deterministic caching, multi-layer support, and hard process isolation verified via host-level leakage tests.